

RECONPRO

FORENSIC CAPABILITY VERIFICATION

Omega

READ-ONLY FORENSIC AUDIT

No code modified. No fixes applied. No implementations created.

Source code is the sole authority. All claims verified against actual repository files.

Generated: 2026-08-13

Repository: /home/z/my-project/src (Next.js 15 App Router)

Audit Scope: 47 API routes, 44 UI components, 16 lib modules, 22 test files

TABLE OF CONTENTS

- 1. Repository Identity and Statistics
- 2. Actual Architecture Map
- 3. Reconnaissance Capability Audit (12 Capabilities)
- 4. Security Assessment Audit (12 Capabilities)
- 5. Specialized Module Audit (10 Modules)
- 6. Team / Organization Capability Audit
- 7. Intelligence Pipeline Audit
- 8. Value Ledger Audit
- 9. Engineering / Security Hardening Audit (20 Claims)
- 10. Testing Verification
- 11. Capability Wiring Analysis
- 12. Dead-Code / False-Capability Detection
- 13. Master Capability Matrix
- 14. Capability Maturity Scores
- 15. Critical Discoveries
- 16. Implemented vs. Partial vs. Future
- 17. Launch Blockers and Recommended Priorities

1. Repository Identity and Statistics

The canonical ReconPro repository is located at `/home/z/my-project/`. It is a Next.js 15 application using the App Router architecture, written in TypeScript, styled with Tailwind CSS 4 and shadcn/ui components, with Framer Motion for animations. The database layer uses Prisma ORM with SQLite as the data store. The project also contains two Python CLI packages (vibesec-cli v8.0.0 and reconpro-work v10.0.0) that represent a separate, legacy codebase and are not part of the Next.js web application under audit. This verification focuses exclusively on the Next.js web application source code in the `src/` directory.

Metric	Value
Framework	Next.js 16.1.1 (App Router)
Language	TypeScript (strict)
Styling	Tailwind CSS 4 + shadcn/ui + Framer Motion
Database	Prisma 6.11.1 + SQLite
Source Files (src/)	162 TypeScript files
API Routes	47 route files across 39 endpoint groups
UI Components	44 ReconPro components + 40 shadcn/ui primitives
Library Modules	16 files in src/lib/ (8 recon, 8 engines)
Test Files	22 test files with ~339 test blocks
Design System	OLED Void: #000000, glass morphism, champagne gold #C9A96E
Python CLI (excluded)	vibesec-cli v8.0.0, reconpro-work v10.0.0
Total JS Estimate	~1.4MB JS, ~340KB CSS (from prior session)

The directory structure is organized into a standard Next.js App Router layout. The `src/app/api/` directory contains 47 route handler files organized into 39 functional endpoint groups, each in its own subdirectory. The `src/lib/` directory contains shared utility libraries, security modules, and reconnaissance engines. The `src/components/reconpro/` directory contains 44 custom UI components that implement the dashboard interface. The `src/lib/recon/` subdirectory contains 8 reconnaissance engine files that implement DNS, HTTP, SSL, CT log, and port scanning capabilities. Additionally, the repository includes two Python packages (vibesec-cli and reconpro-work) that contain extensive Python implementations of security scanning modules, but these are completely disconnected from the Next.js web application and are not imported, referenced, or used by any part of the TypeScript codebase.

2. Actual Architecture Map

The following architecture map is derived exclusively from verified import chains, API route handler logic, and database queries. Modules marked "NOT IMPLEMENTED" have no working code behind them. Modules marked "SIMULATED" contain real code but produce fabricated data rather than performing actual security analysis against live targets.

Layer	Module	Status	Evidence
ENTRY	Landing Page (/)	MARKETING	home-section.tsx renders HeroSection, FeaturesSection, etc. No API calls
ENTRY	Dashboard	FUNCTIONAL	Multiple components fetch from /api/* routes
MIDDLEWARE	Security Headers	IMPLEMENTED	middleware.ts: CSP, HSTS, X-Frame-Options DENY, CORP, COEP
MIDDLEWARE	API Authentication	PARTIAL	api-protection.ts exists, requireAuth=false on ALL routes
MIDDLEWARE	Rate Limiting	IMPLEMENTED	api-security.ts: in-memory bounded store, used by ~30 routes
MIDDLEWARE	SSRF Protection	IMPLEMENTED	safe-fetch.ts, api-security.ts: DNS resolution + IP blocking
ENGINE	DNS Reconnaissance	IMPLEMENTED	native-dns.ts: real DNS queries via dns.promises
ENGINE	SSL/TLS Analysis	IMPLEMENTED	native-dns.ts analyzeSSLPromise: real TLS handshakes
ENGINE	HTTP Recon	DEAD CODE	recon/http-recon.ts exists but NOT imported by any route
ENGINE	Port Scanning	DEAD CODE	recon/port-check.ts exists but NOT imported by any route
ENGINE	CT Log Query	DEAD CODE	recon/ct-logs.ts exists but NOT imported by any route
ENGINE	Vulnerability Scan	IMPLEMENTED	vuln-scan/route.ts: real TCP probes + banner grabs
ENGINE	Bot Hunter	IMPLEMENTED	bot-hunter/route.ts: real DNSBL + IP reputation queries
ENGINE	Hall of Fame	IMPLEMENTED	hall-of-fame/route.ts: real HTTP probes via safeFetch
ENGINE	AI Advisor	HARDCODED KB	ai-advisor/route.ts: pattern-matched remediation database
ENGINE	Oblivion (Model Red Team)	STUB	oblivion/route.ts: returns all zeros, no real analysis
ENGINE	GORGON (Model Red Team)	IMPLEMENTED	model-redteam/route.ts: 955 lines, real HTTP probes
ENGINE	Fear Index	SIMULATED	fear-index-engine.ts: seeded PRNG, no real data
ENGINE	Quantum Doom Clock	SIMULATED	quantum-doom-engine.ts: synthetic TLS data + math
ENGINE	PQC Vault	SIMULATED	pqc-vault-engine.ts: hardcoded compliance databases
ENGINE	CNI Sentinel	SIMULATED	cni-sentinel-engine.ts: static SCADA/ICS databases
ENGINE	Implosion Simulator	SIMULATED	implosion-engine.ts: IBM data + arithmetic
ENGINE	Broadcast System	SIMULATED	broadcast-engine.ts: real Ed25519 crypto, fake content
ENGINE	Genesis Stamp	IMPLEMENTED	genesis-crypto.ts: real Ed25519 signing + DB storage
ENGINE	NHI Kill Switch	HARDCODED	nhi/route.ts: 12 sample identities, no real cloud scan
ENGINE	Compliance Panel	SIMULATED	compliance/route.ts: rule-based scoring
ENGINE	Sovereign Control	IMPLEMENTED	sovereign/route.ts: real Ed25519 crypto + DB audit

Layer	Module	Status	Evidence
DATA	Prisma ORM	IMPLEMENTED	20 models, SQLite, used by 15+ routes
DATA	Findings Pipeline	PARTIAL	Scan creates Finding records, no correlation engine
DATA	Intelligence Pipeline	NOT IMPLEMENTED	No observation-to-risk-to-remediation flow

3. Reconnaissance Capability Audit

Each reconnaissance capability was traced from UI component through API route to engine library. A capability is classified as VERIFIED IMPLEMENTED only when the complete execution path is connected end-to-end with real network operations. Capabilities that exist as library code but are not imported by any production route are classified as DEAD CODE, regardless of how well-implemented the library itself may be.

	Status	Implementation Files	Evidence	Confidence
con	VERIFIED	native-dns.ts, scan/route.ts	Real DNS A/AAAA/MX/NS/TXT/DMARC queries	HIGH
e	VERIFIED	native-dns.ts, scan/route.ts	SPF/DMARC analysis, TXT classification	HIGH
	VERIFIED	native-dns.ts, scan/route.ts	DNS resolution + reverse DNS via dns.promises	HIGH
a	NOT FOUND	None wired to production	recon/ct-logs.ts exists (DEAD CODE) + scan/stream is simulated	N/A
e	NOT FOUND	No implementation found	No WHOIS query code exists anywhere in src/	N/A
	VERIFIED	native-dns.ts, scan/route.ts	Real TLS handshakes, cert analysis, cipher check	HIGH
ence	DEAD CODE	recon/ct-logs.ts	Full crt.sh implementation exists but is NOT imported	N/A
n	DEAD CODE	recon/http-recon.ts	Full HTTP + security header analysis exists but NOT imported	N/A
on	DEAD CODE	recon/http-recon.ts	30+ entry fingerprint DB exists in dead code	N/A
ing	DEAD CODE	recon/port-check.ts	26-port TCP scanner exists but NOT imported by any route	N/A
	VERIFIED	scan/route.ts	Inline HTTP header analysis in scan route	MEDIUM
face	PARTIAL	hall-of-fame/route.ts	5-path HTTP probe (.env, /admin, etc.) via safeFetch	MEDIUM

CRITICAL FINDING

The `src/lib/recon/` directory containing 6 fully-implemented reconnaissance modules (`http-recon.ts`, `ssl-recon.ts`, `ct-logs.ts`, `port-check.ts`, `ssrf-guard.ts`, `findings-formatter.ts`) is entirely DEAD CODE. These files contain real, working implementations with actual network calls, SSRF protection, and comprehensive analysis. However, NO production API route imports from this directory. The `scan/route.ts` instead uses `native-dns.ts` for DNS/SSL only and implements HTTP analysis inline. Subdomain enumeration, port scanning, CT log querying, and the centralized `ssrf-guard` are completely unreachable.

The production scan route (`src/app/api/scan/route.ts`) is a 1200+ line file that implements DNS enumeration (7 concurrent record types via `native-dns.ts`), SSL/TLS analysis (via `native-dns.ts`), HTTP header analysis (inline `safeFetch` calls), technology fingerprinting (inline ~30-entry database), vulnerability probing (inline CVE matching), and port scanning (inline TCP probes of 13 common ports). It saves results to the database via Prisma. The `scan/stream/route.ts` is a separate SSE endpoint that provides a simulated terminal experience with fake command outputs and hardcoded phase transitions, not connected to the actual scan engine.

4. Security Assessment Audit

status	Implementation Details	Confidence
VERIFIED	vuln-scan/route.ts: TCP banner grab, CVE matching, HTTP/SSL/DNS tests	HIGH
VERIFIED	hall-of-fame/route.ts: 5-path HTTP exposure probe	MEDIUM
VERIFIED	api-security.ts: domain/IP validation, SSRF, rate limiting	HIGH
VERIFIED	safe-fetch.ts: DNS resolution, private IP blocking, domain blocklist	HIGH
VERIFIED	middleware.ts: CSP, HSTS, X-Frame-Options, CORP, COEP	HIGH
VERIFIED	hall-of-fame/route.ts + vuln-scan: real HTTP probing	MEDIUM
VERIFIED	scan/route.ts: severity/category assignment per finding	MEDIUM
PARTIALSEC	scan/route.ts: severity assignment; no CVSS scoring engine	LOW
VERIFIED	scan/route.ts: evidence field populated from actual output	HIGH
PARTIALSEC	Prisma Finding model; inconsistent use across routes	LOW
PARTIALSEC	Dead code findings-formatter.ts unused; inline formatting	LOW
NOT FOUND	No report generation engine; data returned as JSON	N/A

The security assessment capabilities are concentrated in the scan and vuln-scan API routes. The vulnerability scanner ([vuln-scan/route.ts](#)) is a 56KB file implementing real TCP connection probes, banner grabbing, HTTP security header checks, SSL certificate validation, DNS misconfiguration detection, and CVE database matching. It uses the centralized security modules (api-security.ts for validation, safe-fetch.ts for HTTP, api-protection.ts for rate limiting). The Hall of Fame route performs 5-path HTTP exposure probing against real targets. However, there is no unified security report generation engine, no CVSS scoring calculator, and the findings-formatter.ts utility in the dead code directory is not used by any production route.

5. Specialized Module Audit

Module	Exists?	Real or Demo?	API Route?	UI Component?	DB?	Tests?	Maturity
Vulnerability Scanner	YES	REAL	vuln-scan/	vuln-arsenal.tsx	Finding, Scan	scan-engine.test (mocked)	PRODUCTION
Bot Hunter	YES	REAL	bot-hunter/	bot-cage.tsx	ThreatAlert	scan-engine.test (mocked)	PRODUCTION
Oblivion	YES	STUB/ZERO	oblivion/	oblivion.tsx	None	None	PLACEHOLDER
Model Red Team (GORGON)	YES	REAL	model-redteam/	model-breaker.tsx	None	None	PRODUCTION
Hall of Fame	YES	REAL	hall-of-fame/	hall-of-fame.tsx	VibeSecEntry	None	PRODUCTION
Sovereign	YES	REAL	sovereign/	sovereign-control.tsx	None	None	PRODUCTION
Sandbox	YES	DEMO	sandbox/	confused-deputy.tsx	None	None	DEMO
Scan/Stream	YES	SIMULATED	scan/stream/	scan-overlay.tsx	None	None	SIMULATION
Recon Engine	PARTIALS SEC	REAL+DEAD	scan/	unified-cli.tsx	Scan, Finding	Mocked tests	PARTIAL
Findings Infra	YES	REAL	scans/	scan-results.tsx	Finding model	None	PRODUCTION

OBLIVION IS A STUB

The Oblivion module ([src/app/api/oblivion/route.ts](#)) presents an elaborate 20-tool analytical dissolution framework with philosophical descriptions for each tool (Cognitive Mirror, Theseus Test, Alignment Decay Engine, etc.). However, the POST handler returns ALL ZEROS for every metric: threatScore=0, bypasses=0, decaysAchieved=0, layersStripped=0, etc. The comment explicitly states "Python engine replaced with structured response" and "awaiting deployment." The 44-tool catalog and wisdom quotes are hardcoded. This is a presentation shell with no actual analysis engine behind it.

GORGON (Model Red Team, [model-redteam/route.ts](#)) is a substantial 955-line implementation that performs actual HTTP probes against AI model endpoints. It sends crafted payloads (prompt injection chains, system prompt extraction attempts, alignment probing), analyzes responses, and matches findings against a CVE database. It uses safeFetch for SSRF protection and rate limiting via api-security.ts. The Hall of Fame module performs real HTTP HEAD requests to probe for exposed paths (.env, /admin, /api/webhooks, /dashboard, /uploads/) and computes a security grade based on exposure results. The Sovereign module uses real Ed25519 cryptographic signing for data sovereignty attestation.

6. Team / Organization Capability Audit

The team and organization capabilities are implemented through a Prisma database schema with 4 models (Organization, Team, Member, TeamMember) and 3 API routes (teams/, members/, organization implied). The schema supports multi-organization architecture with teams, role-based membership (owner, admin, security_lead, analyst, viewer), and audit logging. However, the actual execution path reveals significant gaps.

Layer	Implementation	Status	Evidence
Authentication	withProtection(requireAuth)	NOT ACTIVE	requireAuth=false on ALL routes; no route checks auth
API Key System	api-protection.ts + ApiKey model	IMPLEMENTED	SHA-256 key hashing, DB lookup, expiry, usage tracking
API Key Usage	Production routes	NOT USED	No route has requireAuth=true; keys exist but unused
Rate Limiting	api-security.ts checkRateLimit	IMPLEMENTED	Bounded in-memory store (50K cap), used by 30+ routes
Input Validation	api-security.ts sanitizeDomain	IMPLEMENTED	Domain regex, IP validation, blocked domain list
Request Size Limit	api-security.ts parseValidatedBody	IMPLEMENTED	1MB default max, Content-Length check
Error Isolation	safeErrorResponse	IMPLEMENTED	Dev: full error; Prod: generic message + UUID
Audit Logging	Prisma AuditLog model	PARTIALSEC	Only teams/ and members/ routes write audit logs
Authorization	Role-based access	NOT IMPLEMENTED	Member roles exist in schema but no route checks roles
Organization Multi-tenancy	organizationId foreign key	PARTIALSEC	Schema supports it; routes use findFirst (single org)

AUTHENTICATION IS NOT ENFORCED

The centralized protection middleware (api-protection.ts) has a fully implemented authentication system with API key hashing, database lookup, expiry checking, and usage tracking. However, the `requireAuth` parameter defaults to `false` and NO production route passes `requireAuth=true`. This means ANY anonymous user can invoke ANY API endpoint including scan creation, team management, member operations, and data deletion. The API key infrastructure exists but is completely dormant.

The request execution path for a typical mutating operation (e.g., DELETE /api/teams?id=xxx) flows through: rate limiting (IP-based, 5 req/min) to route handler to Prisma database operations. There is NO authentication check, NO authorization check, and NO organization scoping (uses findFirst which returns any organization). The audit log is written for team creation and deletion but not for all mutating operations. The DELETE operation for teams does cascade-delete TeamMember junction records, which shows reasonable data integrity handling.

7. Intelligence Pipeline Audit

Stage	Implemented?	Location	Data Structure	Connected to Next?
Raw Recon	YES	scan/route.ts	Finding type (inline)	YES -> DB
Observations	YES	scan/route.ts	Finding with severity/category	YES -> DB
Findings	YES	Prisma Finding model	Finding table with status/CVE/CVSS	NO -> No consumer
Correlation	NO	None	N/A	N/A
Risk	PARTIALSEC	scan/route.ts riskScore	Integer 0-100	Stored, not calculated
Business Impact	NO	None	N/A	N/A
Remediation	PARTIALSEC	ai-advisor hardcoded KB	Static text responses	NOT from findings
Verification	NO	None	N/A	N/A

The intelligence pipeline is implemented only at its earliest stages. The scan route produces raw reconnaissance observations and converts them into findings with severity and category classifications. These findings are stored in the Prisma Finding table with fields for title, severity, category, description, evidence, asset, remediation, CVE, CVSS, and status. However, the pipeline breaks after storage. There is no correlation engine that connects multiple findings into a larger security picture. Risk scores are stored as static integers on the Scan model but are not dynamically calculated from findings. Business impact modeling does not exist (the Implosion Simulator produces fabricated financial estimates from hardcoded industry averages, not from actual scan data). Remediation guidance exists only as a static keyword-matching database in the AI Advisor route. There is no verification mechanism to track whether a previously identified issue has been resolved.

8. Value Ledger Audit

Capability	Status	Evidence
Estimated Exposure	SIMULATED	implosion-engine.ts: IBM Cost of Data Breach 2024 data + arithmetic
Potential Financial Impact	SIMULATED	implosion-engine.ts: hardcoded regulatory fines (GDPR/HIPAA/PCI)
Avoided Loss	NOT FOUND	No implementation exists
Remediation Value	NOT FOUND	No implementation exists
Business Value Tracking	NOT FOUND	No implementation exists
Verified Savings	NOT FOUND	No implementation exists
Value Attribution	NOT FOUND	No implementation exists
Customer-Specific Tracking	PARTIALSEC	Organization model exists but no value metrics
Financial Reporting	NOT FOUND	No implementation exists

The Value Ledger is almost entirely a product vision with no production implementation. The Implosion Simulator is the only component that addresses financial metrics, but it operates on hardcoded IBM research data and industry averages rather than actual organizational telemetry. When a user provides domain and industry parameters, the simulator generates plausible-looking but entirely synthetic financial impact estimates (data breach cost, regulatory fines, reputational damage, stock impact). The ImplosionScenario Prisma model can store these simulations, but there is no mechanism for tracking remediation value, verified savings, or customer-specific value attribution. The Organization model has fields for plan type and API quota but no financial or value-related fields.

9. Engineering / Security Hardening Audit

Claim	Status	Evidence	Notes
SSRF Protection	VERIFIED	safe-fetch.ts + api-security.ts	DNS resolution, IPv4/IPv6 private blocking, domain blocklist
Centralized Validation	VERIFIED	api-security.ts	sanitizeDomain, sanitizeTarget, isValidTarget, DOMAIN_REGEX
API Authentication	IMPLEMENTED BUT UNUSED	api-protection.ts withProtection	Full implementation exists but requireAuth=false everywhere
API Key Protection	IMPLEMENTED BUT UNUSED	api-protection.ts + ApiKey model	SHA-256 hashing, DB lookup, expiry; no route uses it
Rate Limiting	VERIFIED	api-security.ts checkRateLimit	Bounded store (50K), cleanup, per-IP keys
Safe Network Fetching	VERIFIED	safe-fetch.ts	SSRF + timeout + size limit + no redirect follow
XSS Escaping	VERIFIED	lib/utils.ts escapeHtml	Function exists but NOT verified in component rendering paths
CSP/Security Headers	VERIFIED	middleware.ts	CSP with nonce, HSTS 1yr, X-Frame DENY, CORP/COEP
Error Isolation	VERIFIED	safeErrorResponse	Dev: full error; Prod: generic + UUID
Error Leak Reduction	VERIFIED	safeErrorResponse	isDev flag controls detail exposure
Accessibility	MINIMAL	Partial ARIA in CommandPalette	No systematic a11y audit; skip-link exists
Security Regression Tests	VERIFIED	22 test files, ~339 tests	Heavy mocking; 38% are static grep tests
Adversarial Tests	VERIFIED	adversarial-ssrf, adversarial-xss, etc.	Good coverage of api-security functions
Red-Team Tests	NOT FOUND	No dedicated red-team test suite	Chaos-forge exists but tests mocks, not real system
Architecture Refactoring	VERIFIED	child_process eliminated	All shell exec replaced with native Node.js APIs
No child_process	VERIFIED	grep: 0 runtime uses	Only comment references ("eliminated during Biological Forge")
No eval()	VERIFIED	grep: 0 in src/	No eval() calls in any source file
No Dangerous Raw Fetch	VERIFIED	All HTTP via safe-fetch.ts	safeFetch used in scan, vuln-scan, bot-hunter, hall-of-fame
TypeScript Strictness	VERIFIED	tsconfig.json strict: true	No implicit any, strict null checks enabled
Production Build Config	VERIFIED	next.config.ts	removeConsole: true, reactStrictMode, output: standalone

10. Testing Verification

The test suite uses Vitest 4.1.10 with jsdom environment. Testing-library/react and testing-library/jest-dom are installed but **never used** in any test file. No "test" script exists in package.json, meaning tests cannot be run via npm test or bun test and are not wired into any CI pipeline. The 22 test files contain approximately 339 test blocks (it() calls).

Category	Files	Tests	% of Total	Assessment
Real Unit Tests	10 files	~195	57%	Test actual production imports
Static Grep Tests	8 files	~130	38%	Read files as text, check patterns
Copied Production Code	1 file	6	2%	api-security.test.ts copies code inline
Tautological Tests	2 files	~8	2%	expect(true).toBe(true) always passes
TOTAL	22 files	~339	100%	

ZERO API ROUTE TESTS

There are **47 API route files** in the codebase. **NONE are tested**. No test imports or invokes any API route handler. No test verifies that GET /api/scan validates input, that POST /api/nhi/revoke checks permissions, or that any endpoint returns correct status codes. The test suite exercises api-security.ts functions extensively (8+ files, 150+ tests) but never tests the routes that use them.

HEAVY MOCKING

The scan-engine, chaos-forge, mutation-forge, and adversarial test files mock **dns/promises, tls, net, native-dns, and globalThis.fetch simultaneously**. The DNS function tests set a mock return value and immediately assert that return value, which is a circular mock round-trip that tests nothing about the actual implementation. The SSRF tests via safeFetch never touch a real DNS resolver or real HTTP server.

api-security.test.ts COPIES CODE

This file duplicates isPrivateIP(), DOMAIN_REGEX, and BLOCKED array **inline into the test file** rather than importing from @/lib/api-security. Tests pass against the copied code, not the actual module. If production code changes, these tests continue to pass against the old copy, creating complete false confidence.

11. Capability Wiring Analysis

Each claimed capability was traced through the full execution path: User Action -> UI -> API -> Validation -> Engine -> Data -> Result -> UI/Report. A capability is fully wired only when every layer in this chain is connected with actual logic (not stubs, mocks, or hardcoded returns).

Capability	UI -> API	API -> Engine	Engine -> Data	Data -> UI	Verdict
Domain/SSL Scan	VERIFIED	VERIFIED	VERIFIED (DB)	VERIFIED	FULLY WIRED
Vulnerability Scan	VERIFIED	VERIFIED	VERIFIED (DB)	VERIFIED	FULLY WIRED
Bot Hunter	VERIFIED	VERIFIED	NOT PERSISTED	VERIFIED	PARTIALLY WIRED
Hall of Fame	VERIFIED	VERIFIED	VERIFIED (DB)	VERIFIED	FULLY WIRED
Model Red Team	VERIFIED	VERIFIED	NOT PERSISTED	VERIFIED	PARTIALLY WIRED
Fear Index	VERIFIED	SIMULATED	NOT PERSISTED	VERIFIED	SIMULATED PIPELINE
Doom Clock	VERIFIED	SIMULATED	NOT PERSISTED	VERIFIED	SIMULATED PIPELINE
PQC Vault	VERIFIED	SIMULATED	NOT PERSISTED	VERIFIED	SIMULATED PIPELINE
CNI Sentinel	VERIFIED	SIMULATED	NOT PERSISTED	VERIFIED	SIMULATED PIPELINE
Oblivion	VERIFIED	STUB (zeros)	NOT PERSISTED	VERIFIED	STUB PIPELINE
NHI Kill Switch	VERIFIED	HARDCODE D	VERIFIED (DB)	VERIFIED	SEED DATA ONLY
Team Management	VERIFIED	VERIFIED	VERIFIED (DB)	VERIFIED	FULLY WIRED
Compliance Panel	VERIFIED	SIMULATED	NOT PERSISTED	VERIFIED	SIMULATED PIPELINE
Genesis Stamp	VERIFIED	VERIFIED	VERIFIED (DB)	VERIFIED	FULLY WIRED
Sovereign Control	VERIFIED	VERIFIED	NOT PERSISTED	VERIFIED	PARTIALLY WIRED
Scan Stream (SSE)	VERIFIED	SIMULATED	NOT PERSISTED	VERIFIED	FAKE TERMINAL

12. Dead-Code / False-Capability Detection

Item	Type	Impact	Evidence
src/lib/recon/* (6 files)	DEAD CODE	HIGH	600+ lines of real recon logic (HTTP, SSL, CT, port) never imported
src/lib/recon/ssrf-guard.ts	DEAD CODE	MEDIUM	Centralized SSRF guard written but unused by sibling modules
src/lib/recon/findings-for-matter.ts	DEAD CODE	LOW	Finding factory unused; inline types used instead
scan/stream/route.ts	FAKE TERMINAL	MEDIUM	SSE endpoint emits hardcoded phase transitions, not real scan data
scan-overlay.tsx	FAKE PROGRESS	LOW	Math.random() progress bar + 10 hardcoded "live findings"
live-terminal.tsx	FAKE TERMINAL	LOW	30 command templates filled with random IPs, not real output
matrix-terminal.tsx	FAKE SCAN	MEDIUM	5 demo targets with hardcoded findings incl. fake CVEs
data/content.ts	MARKETING DATA	LOW	All hardcoded stats (2847 scans, \$2.4B protected, etc.)
fear-index-engine.ts	SIMULATED ENGINE	MEDIUM	Seeded PRNG generates fake threat scores presented as real intelligence
quantum-doom-engine.ts	SYNTHETIC DATA	MEDIUM	generateSyntheticTLSData() creates fake TLS from domain hash
broadcast-engine.ts	FABRICATED DATA	MEDIUM	10 demo bulletins with fake CVEs and fake threat actor names
nhi SAMPLE_IDENTITIES	SEED DATA	MEDIUM	12 hardcoded cloud identities, no real cloud API scanning

The most significant dead-code finding is the entire `src/lib/recon/` directory (8 files, estimated 1500+ lines). These files contain fully implemented, production-quality reconnaissance modules with real network calls, SSRF protection, comprehensive analysis, and proper error handling. They were clearly written as a refactored reconnaissance library intended to replace the inline code in `scan/route.ts`. However, the migration was never completed. The scan route continues to use `native-dns.ts` for DNS/SSL and implements HTTP analysis inline, while the `recon/` modules sit unused. Additionally, the simulated engines (`fear-index`, `quantum-doom`, `pqc-vault`, `cni-sentinel`, `broadcast`) generate impressive-looking but entirely fabricated security intelligence that could mislead users into believing they are viewing real data.

13. Master Capability Matrix

Capability	Status	Files	API	UI	Tests	Integration	Limitations	Confidence
DNS Reconnaissance	VERIFIED	native-dns.ts	scan/	unified-cli	mocked	DB storage	No subdomain enum	HIGH
SSL/TLS Analysis	VERIFIED	native-dns.ts	scan/	unified-cli	mocked	DB storage	Chain length=0	HIGH
HTTP Header Analysis	VERIFIED	scan/route.ts	scan/	unified-cli	mocked	DB storage	No redirect follow	MEDIUM
Vulnerability Scanning	VERIFIED	vuln-scan/route.ts	vuln-scan/	vuln-arsenal	mocked	DB storage	13 ports only	HIGH
Bot/C2 Detection	VERIFIED	bot-hunter/route.ts	bot-hunter/	bot-cage	mocked	Not persisted	Limited DNSBL	MEDIUM
Security Grading	VERIFIED	hall-of-fame/route.ts	hall-of-fame/	hall-of-fame	none	DB storage	5 probe paths	MEDIUM
Model Red Team	VERIFIED	model-red-team/route.ts	model-redteam/	model-brake	none	Not persisted	955 lines, real probes	HIGH
AI Advisor	HARDCODED	ai-advisor/route.ts	ai-advisor/	ai-advisor	none	Static KB	Keyword matching only	LOW
Fear Index	SIMULATED	fear-index-engine.ts	fear-index/	fear-index	indirect	Not persisted	Seeded PRNG data	N/A
Doom Clock	SIMULATED	quantum-doom-engine.ts	doom-clock/	doom-clock	indirect	Not persisted	Synthetic TLS data	N/A
PQC Vault	SIMULATED	pqc-vault-engine.ts	pqc-vault/	pqc-vault	none	Not persisted	Static compliance DB	N/A
CNI Sentinel	SIMULATED	cni-sentinel-engine.ts	cni-sentinel/	cni-sentinel	none	Not persisted	Static ICS/SCADA DB	N/A
Implosion Sim	SIMULATED	implosion-engine.ts	implosion/	implosion-panel	none	DB storage	IBM data + arithmetic	N/A
Oblivion	STUB	oblivion/route.ts	oblivion/	oblivion	none	None	All zeros returned	N/A
NHI Kill Switch	SEED DATA	nhi/route.ts	nhi/	nhi-kill-switch	none	DB storage	12 hardcoded identities	LOW
Genesis Stamp	VERIFIED	genesis-crypto.ts	genesis/	genesis-stamp	none	DB+crypto	Ed25519 attestation	HIGH
Sovereign Control	VERIFIED	sovereign-crypto.ts	sovereign/	sovereign-ctrl	none	DB+crypto	Ed25519 signing	HIGH
Broadcast System	SIMULATED	broadcast-engine.ts	broadcast/	broadcast-ctr	none	In-memory only	Fake demo bulletins	N/A
Compliance Panel	SIMULATED	compliance/route.ts	compliance/	compliance	none	Not persisted	Rule-based scoring	N/A

Capability	Status	Files	API	UI	Tests	Integration	Limitations	Confidence
Team Management	VERIFIED	teams/, members/	teams/	team-mgmt	none	DB+audit	No auth/roles enforced	MEDIUM
Monitoring Policies	VERIFIED	monitoring/	monitoring/	monitor-panel	none	DB storage	No scheduler execution	MEDIUM
Scan History	VERIFIED	scans/routes.ts	scans/	war-room	none	DB read	Last 20 scans only	MEDIUM
Exec Dashboard	SIMULATED	executive/route.ts	executive/	ceo-dashboard	none	Not persisted	Hardcoded metrics	N/A

14. Capability Maturity Scores

Each domain is scored 0-10 based on verified evidence only. Scores reflect what the repository ACTUALLY implements, not what is planned, declared, or simulated. Simulated capabilities receive zero credit.

Domain	Score	Justification
Reconnaissance	5/10	Real DNS/SSL/HTTP/port scanning in production; but subdomain, CT logs, WHOIS missing; 6 dead-code modules
Security Assessment	5/10	Real vuln scanning, bot detection, security grading; no CVSS scoring, no report generation, no correlation
Specialized Modules	3/10	Oblivion=stub, GORGON=real, Hall of Fame=real, NHI=seed data, Sandbox=demo, Fear/Doom/PQC/CNI=simulated
Team/Organization	4/10	Full CRUD with DB+audit; no auth enforcement, no roles, no multi-tenancy
Intelligence Pipeline	2/10	Raw recon -> findings stored; no correlation, risk calc, business impact, remediation tracking
Reporting	1/10	No report generation; findings returned as JSON; no PDF/HTML report builder
Value Tracking	0/10	Not implemented; Implosion simulates financial data from hardcoded averages
Security Hardening	7/10	SSRF, CSP, HSTS, rate limiting, input validation, error isolation, no child_process, no eval
Testing	3/10	339 tests exist; 38% are grep tests; 0 API route tests; heavy mocking; no CI; no test script
Production Readiness	4/10	Build passes; no auth; no multi-tenancy; SQLite not production-grade; no CI/CD

**OVERALL EVIDENCE-DERIVED
CAPABILITY MATURITY SCORE**

3.4 / 10

15. Critical Discoveries

15.1 Verified Strengths

The SSRF protection system is genuinely impressive. The `safe-fetch.ts` module implements a multi-layered defense: domain blocklist checking, IPv4/IPv6 private range blocking (covering RFC 1918, loopback, link-local, carrier-grade NAT, cloud metadata, multicast, and reserved ranges), DNS resolution before connection with DNS-failure-treated-as-unsafe semantics, and response size limiting. The `api-security.ts` module provides centralized input validation with strict domain regex, comprehensive blocked domain list, and bounded in-memory rate limiting with automatic cleanup. The security middleware applies defense-in-depth headers (CSP with nonce, HSTS 1 year, X-Frame-Options DENY, Cross-Origin policies) to all routes. The elimination of `child_process` is verified: all network operations use native Node.js APIs (`dns.promises`, `tls.connect`, `net.Socket`). The Prisma schema is well-designed with 20 models covering organizations, teams, scans, findings, compliance, monitoring, integrations, API keys, audit logs, NHI identities, and Genesis stamps.

15.2 False / Exaggerated Capabilities

- Oblivion Model Red Team: Presented as a "20-tool analytical dissolution framework" but returns all zeros. The POST handler explicitly states it is "awaiting deployment." The 20 tools, wisdom quotes, and philosophical descriptions create a convincing facade of capability that does not exist.
- Fear Index: Presented as a "CISO Fear Index" with sector breakdowns and historical trends. In reality, it generates deterministic numbers from a seeded PRNG based on the current date. The threat descriptions are pre-written strings.
- Quantum Doom Clock: Presented as a quantum threat assessment. In reality, when no real TLS data is provided (which is the normal case), it fabricates synthetic TLS data from a hash of the domain name and performs arithmetic on hardcoded quantum milestones.
- CNI Sentinel: Presented as a critical infrastructure threat analyzer. In reality, it performs rule-based lookups against hardcoded SCADA protocol databases, APT group lists, and firmware CVE databases. No actual network analysis occurs.
- Scan Stream (SSE): Presented as a live scan terminal. In reality, it emits hardcoded phase transitions with fake command outputs (`dig`, `curl`, `openssl` strings) at random intervals, regardless of whether a real scan is running.

15.3 Partial Capabilities

- Intelligence Pipeline: Raw recon to findings storage works. Everything after that (correlation, risk calculation, business impact, remediation, verification) is missing.
- Authentication: Full API key infrastructure exists (SHA-256 hashing, DB lookup, expiry, scopes) but is completely dormant because no route enforces it.
- NHI Kill Switch: DB schema and CRUD operations work, but identity discovery uses 12 hardcoded sample identities rather than actual cloud API scanning.
- Monitoring Policies: Model supports scheduling (hourly, daily, weekly, monthly) but no scheduler executes these policies automatically.

15.4 Critical Gaps

- **NO AUTHENTICATION ENFORCED:** Any anonymous user can invoke any API endpoint including mutating operations (team creation, deletion, NHI revocation).
- **ZERO API ROUTE TESTS:** 47 API routes with zero test coverage.
- **NO CI/CD:** No test script in package.json, no CI configuration, no deployment pipeline.
- **SQLITE NOT PRODUCTION-GRADE:** Single-file SQLite cannot support concurrent users, replication, or the scale implied by the enterprise schema.
- **IN-MEMORY STATE:** Rate limiting, broadcast storage, and fear index state are lost on server restart.
- **SIMULATED ENGINES PRESENTED AS REAL:** 5 engines (fear-index, quantum-doom, pqc-vault, cni-sentinel, broadcast) generate fabricated data without clear disclosure to users.

15.5 Architectural Risks

The single-organization assumption in all API routes (using `findFirst` instead of organization-scoped queries) means the multi-tenant schema is decorative. The in-memory rate limiting store (bounded to 50K entries) will lose all state on server restart, allowing rate limit evasion. The broadcast system uses an in-memory Map that evaporates on restart, making all Ed25519 signatures unverifiable (new keypair generated each restart). The MonitoringPolicy model has schedule fields but no scheduler implementation, meaning scheduled scans will never execute.

15.6 Security Risks

- Team/Member deletion without auth: `DELETE /api/teams` and `DELETE /api/members` are protected only by rate limiting (5 req/min). Any client can delete all teams and members.
- NHI revocation without auth: `POST /api/nhi/revoke` can revoke non-human identities without any authentication.
- Genesis stamp issuance without auth: `POST /api/genesis` can issue cryptographic attestations without authentication.
- Broadcast creation without auth: `POST /api/broadcast` can create cryptographically signed security bulletins without authentication.
- native-dns.ts has no internal SSRF protection: The `scan/route.ts` relies on inline validation before calling native-dns functions, but native-dns itself does not validate targets.

16. Implemented vs. Partial vs. Future

16.1 ALREADY REAL

Capabilities genuinely implemented, connected, and producing real results from actual operations:

- DNS/SSL/TLS reconnaissance with real network queries and database persistence
- HTTP security header analysis with technology fingerprinting
- Vulnerability scanning with TCP port probing and banner grabbing
- Bot/C2 detection with DNSBL and IP reputation queries
- Security grading with exposure probing
- GORGON model red-teaming with actual HTTP probes
- Hall of Fame with real HTTP exposure testing
- Genesis Stamp with Ed25519 cryptographic attestation
- Sovereign Control with Ed25519 signing
- Team/Member CRUD with database and audit logging
- Monitor Policy CRUD with database persistence
- SSRF protection system (safe-fetch + api-security)
- Security middleware (CSP, HSTS, X-Frame-Options, CORP, COEP)
- Rate limiting (bounded in-memory store)
- Centralized input validation

16.2 PARTIALLY REAL

Capabilities with meaningful implementation but critical missing pieces:

- Authentication infrastructure: Full implementation exists but is dormant (requireAuth=false everywhere)
- Intelligence pipeline: Recon-to-findings works; correlation/risk/remediation/verification missing
- NHI Kill Switch: Schema and CRUD work; identity discovery uses hardcoded samples
- Monitoring: Model supports scheduling; no scheduler executes policies
- Compliance scoring: Rule-based computation; no real compliance framework integration
- AI Advisor: Static remediation knowledge base; no LLM integration

16.3 FUTURE RECONPRO

Capabilities that belong to the product vision but have no current implementation:

- Subdomain enumeration (dead code exists but not wired)
- CT log analysis (dead code exists but not wired)
- Port scanning beyond 13 ports (dead code with 26 ports exists but not wired)
- WHOIS intelligence
- Finding correlation engine
- Dynamic risk scoring

- Business impact modeling from actual data
- Remediation tracking and verification
- Security report generation (PDF/HTML)
- Value attribution and financial reporting
- Real multi-tenant organization scoping
- Role-based access control enforcement
- CI/CD pipeline and automated testing
- Real-time scheduled scan execution
- Integration with external SIEM/SOAR platforms
- Webhook notifications
- Real OBLIVION model red-team analysis

17. Launch Blockers and Recommended Priorities

17.1 Hard Launch Blockers

These issues MUST be resolved before any production deployment:

#	Blocker	Severity	Effort
1	No authentication on ANY API route	CRITICAL	Medium
2	No authorization/role enforcement	CRITICAL	Medium
3	SQLite not suitable for production	CRITICAL	Large
4	Simulated engines presented without disclosure	HIGH	Small
5	No CI/CD pipeline	HIGH	Medium
6	Zero API route test coverage	HIGH	Large
7	In-memory state lost on restart	HIGH	Medium

17.2 Recommended Engineering Priorities

In priority order, these are the highest-impact improvements that would most advance ReconPro toward a production-viable security platform:

Priori ty	Task	Impact	Details
P0	Activate authentication	CRITICAL	Set requireAuth=true on all mutating routes; add public-only scan route
P0	Migrate to PostgreSQL	CRITICAL	Replace SQLite; enables concurrent users, replication, production scale
P1	Add API route tests	HIGH	Cover all 47 routes with at least basic input validation and auth tests
P1	Wire dead recon modules	HIGH	Connect recon/http-recon, recon/ct-logs, recon/port-check to scan route
P1	Disclose simulated engines	HIGH	Mark fear-index, doom-clock, pqc-vault, cni-sentinel as "demo/simulated" in UI
P2	Implement OBLIVION	MEDIUM	Either build real model red-team or remove the stub to avoid false impression
P2	Replace NHI seed data	MEDIUM	Implement real cloud API scanning or clearly label as demo data
P2	Add organization scoping	MEDIUM	Replace findFirst with authenticated organization context
P3	Build correlation engine	MEDIUM	Connect findings to risk scoring, business impact modeling
P3	Implement scheduled scans	LOW	Build scheduler for MonitorPolicy execution
P3	Generate security reports	LOW	Build PDF/HTML report generation from findings data
P4	Add CI/CD pipeline	LOW	GitHub Actions or equivalent; test script in package.json; build verification

